ORQLAND Real-Time Chat Platform

Complete Relational Database Specification, DDL Schema, Queries & Performance Indexes

Database Engine: PostgreSQL 16 (Drizzle ORM & Raw SQL)

High Availability: Redis PubSub Cluster + Socket.IO Adapter

Retention Policy: 48-Hour TTL Automated Media Lifecycle

1. System Architecture & Relational Topology

The ORQLAND Chat architecture enforces strict relational integrity, single active chat sessions per client-business pairing, idempotent sequential messaging, and offloaded 48-hour presigned object storage for multimedia assets.

Transactional Core (PostgreSQL 16)

Guarantees ACID transactions for message sequencing, atomic unread count increments, participant read receipts, and outbox event dispatching.

Distributed PubSub (Redis 7)

Provides multi-pod horizontal scalability across WebSocket gateways with sub-5ms ephemeral presence states and typing indicators.

2. Entity: `users` Table

Stores account identity mapped from Firebase Auth UIDs to internal UUID identifiers with user roles (`client` or `business`).

Column	Data Type	Constraints	Description / Purpose
`id`	`UUID`	PK `gen_random_uuid()`	Internal immutable primary key for database relations
`firebase_uid`	`VARCHAR(128)`	`NOT NULL, UNIQUE`	External Firebase Authentication UID mapping
`email`	`VARCHAR(255)`	`NOT NULL, UNIQUE`	Normalized lowercase user contact email
`name`	`VARCHAR(255)`	`NOT NULL`	Full user or business profile display name
`handle`	`VARCHAR(100)`	`NULL`	Unique profile username handle (e.g. `@sidharth`)
`avatar_url`	`TEXT`	`NULL`	Avatar CDN URL or public profile photo link
`role`	`VARCHAR(20)`	`DEFAULT 'client'`	System role: `'client'` or `'business'`
`business_status`	`VARCHAR(20)`	`DEFAULT 'Open'`	Operating status for businesses (`'Open'` | `'Closed'`)
`is_online`	`BOOLEAN`	`DEFAULT FALSE`	Persistent fallback for real-time presence
`last_seen_at`	`BIGINT`	`NULL`	Unix epoch in milliseconds of user's last socket heartbeat
`created_at`	`TIMESTAMPTZ`	`DEFAULT NOW()`	Account creation timestamp
`updated_at`	`TIMESTAMPTZ`	`DEFAULT NOW()`	Row modification timestamp

DDL Query: Create `users` Table & Indexes

```
CREATE TABLE users (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
```

```
    firebase_uid VARCHAR(128) NOT NULL UNIQUE,
    email VARCHAR(255) NOT NULL UNIQUE,
    name VARCHAR(255) NOT NULL,
    handle VARCHAR(100),
    avatar_url TEXT,
    role VARCHAR(20) NOT NULL DEFAULT 'client',
    location VARCHAR(255),
    wsid VARCHAR(100),
    business_status VARCHAR(20) DEFAULT 'Open',
    is_online BOOLEAN DEFAULT FALSE,
    last_seen_at BIGINT,
    created_at TIMESTAMPTZ NOT NULL DEFAULT clock_timestamp(),
    updated_at TIMESTAMPTZ NOT NULL DEFAULT clock_timestamp()
);

CREATE UNIQUE INDEX users_firebase_uid_idx ON users (firebase_uid);
CREATE UNIQUE INDEX users_email_idx ON users (email);
```

3. Entity: `conversations` Table

Represents 1-on-1 dialogue sessions between clients and businesses. Includes denormalized last message text, timestamp, and independent unread counters for 0ms sidebar rendering without nested message aggregation joins.

Column	Data Type	Constraints	Description / Purpose
`id`	`UUID`	`PK` `gen_random_uuid()`	Unique conversation session identifier
`client_id`	`UUID`	`FK` `users(id) ON DELETE CASCADE`	Client participant reference
`business_id`	`UUID`	`FK` `users(id) ON DELETE CASCADE`	Target business participant reference
`status`	`VARCHAR(20)`	`DEFAULT 'active'`	Session lifecycle: `'active'` or `'ended'`
`ended_at`	`BIGINT`	`NULL`	Timestamp (epoch ms) when session was concluded
`ended_by`	`UUID`	`FK` `users(id)`	User who initiated the termination
`workspace_name`	`VARCHAR(255)`	`DEFAULT 'Orqspace'`	Associated workspace or service category topic
`last_message_text`	`TEXT`	`DEFAULT ''`	Cached preview text of the most recent message
`last_message_timestamp`	`BIGINT`	`DEFAULT 0`	Epoch timestamp (ms) for sorting conversation lists
`unread_by_client`	`INTEGER`	`DEFAULT 0, NOT NULL`	Unread counter badge displayed to client
`unread_by_business`	`INTEGER`	`DEFAULT 0, NOT NULL`	Unread counter badge displayed to business
`created_at`	`TIMESTAMPTZ`	`DEFAULT NOW()`	Session creation timestamp

DDL Query: Create `conversations` Table & Foreign Keys

```
CREATE TABLE conversations (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    client_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    business_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    status VARCHAR(20) NOT NULL DEFAULT 'active',
    ended_at BIGINT,
    ended_by UUID REFERENCES users(id),
    deleted_by_business_at BIGINT,
    deleted_for_client_at BIGINT,
    workspace_name VARCHAR(255) DEFAULT 'Orqspace',
    workspace_type VARCHAR(100) DEFAULT 'Orqspace Info',
    workspace_description TEXT,
    booking_service_name VARCHAR(255),
    booking_status VARCHAR(50),
    last_message_text TEXT DEFAULT '',
    last_message_time VARCHAR(50) DEFAULT '',
    last_message_timestamp BIGINT DEFAULT 0,
    unread_by_client INTEGER NOT NULL DEFAULT 0,
    unread_by_business INTEGER NOT NULL DEFAULT 0,
    created_at TIMESTAMPTZ NOT NULL DEFAULT clock_timestamp(),
    updated_at TIMESTAMPTZ NOT NULL DEFAULT clock_timestamp()
);

CREATE INDEX conv_client_idx ON conversations (client_id);
```

```
CREATE INDEX conv_business_idx ON conversations (business_id);
CREATE INDEX conv_last_msg_idx ON conversations (last_message_timestamp DESC);
-- Enforce single active conversation pair constraint
CREATE UNIQUE INDEX conv_pair_unique_idx ON conversations (client_id, business_id)
    WHERE status = 'active';
```

4. Entity: `conversation_participants` Table

Tracks room membership, permission roles, and persistent read cursor pointers per user.

```
CREATE TABLE conversation_participants (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    conversation_id UUID NOT NULL REFERENCES conversations(id) ON DELETE CASCADE,
    user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    role VARCHAR(20) NOT NULL, -- 'client' | 'business'
    joined_at TIMESTAMPTZ NOT NULL DEFAULT clock_timestamp(),
    left_at TIMESTAMPTZ,
    last_read_message_id UUID,
    last_read_at BIGINT
);

CREATE UNIQUE INDEX conv_participant_unique_idx ON conversation_participants (conversation_id, user_id);
CREATE INDEX conv_participant_user_idx ON conversation_participants (user_id);
```

5. Entity: `messages` Table

Core chat message ledger. Supports end-to-end client idempotency keys, delivery confirmations (`sending` → `sent` → `delivered` → `read`), and soft deletions.

Column	Data Type	Constraints	Description / Purpose
`id`	`UUID`	PK `gen_random_uuid()`	Immutable message identifier
`conversation_id`	`UUID`	FK `conversations(id) ON DELETE CASCADE`	Parent conversation conversation reference
`sender_id`	`UUID`	FK `users(id) ON DELETE CASCADE`	Author of the message
`content`	`TEXT`	`DEFAULT ''`	Text content or attachment caption
`type`	`VARCHAR(20)`	`DEFAULT 'text'`	Payload kind: `'text'`, `'media'`, `'location'`, `'system'`
`idempotency_key`	`VARCHAR(128)`	`NULL`	Client UUID preventing duplicate message generation
`status`	`VARCHAR(20)`	`DEFAULT 'delivered'`	Delivery lifecycle: `'sent'` | `'delivered'` | `'read'`
`sequence_number`	`BIGINT`	`NULL`	Strict monotonically increasing sequence per channel
`created_at`	`BIGINT`	`NOT NULL`	Unix epoch millisecond of message dispatch
`is_edited`	`BOOLEAN`	`DEFAULT FALSE`	True if edited within 15-minute edit window
`deleted_for_everyone`	`BOOLEAN`	`DEFAULT FALSE`	Revocation indicator for all participants

DDL Query: Create `messages` Table & Idempotency Index

```
CREATE TABLE messages (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    conversation_id UUID NOT NULL REFERENCES conversations(id) ON DELETE CASCADE,
    sender_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    content TEXT DEFAULT '',
    type VARCHAR(20) DEFAULT 'text',
    idempotency_key VARCHAR(128),
    status VARCHAR(20) DEFAULT 'delivered',
    sequence_number BIGINT,
    created_at BIGINT NOT NULL,
    timestamp_display VARCHAR(50) NOT NULL,
    date_display VARCHAR(50) DEFAULT 'Today',
    is_edited BOOLEAN NOT NULL DEFAULT FALSE,
    edited_at BIGINT,
    deleted_by_client BOOLEAN NOT NULL DEFAULT FALSE,
    deleted_by_business BOOLEAN NOT NULL DEFAULT FALSE,
    deleted_for_everyone BOOLEAN NOT NULL DEFAULT FALSE
);

CREATE UNIQUE INDEX messages_idempotency_idx ON messages (conversation_id, idempotency_key)
    WHERE idempotency_key IS NOT NULL;
CREATE INDEX messages_conv_created_idx ON messages (conversation_id, created_at ASC);
CREATE INDEX messages_sender_idx ON messages (sender_id);
```

6. Entity: `message_attachments` Table (48-Hour TTL Retention)

Stores metadata and signed object storage keys for uploaded media. The `expires_at` field enforces automated 48-hour expiration purge sweeps.

```
CREATE TABLE message_attachments (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    message_id UUID NOT NULL REFERENCES messages(id) ON DELETE CASCADE,
    type VARCHAR(20) NOT NULL, -- 'image' | 'pdf' | 'location'
    name VARCHAR(255) NOT NULL,
    object_key TEXT NOT NULL,
    url TEXT NOT NULL,
    thumbnail_url TEXT,
    mime_type VARCHAR(100),
    size INTEGER,
    upload_status VARCHAR(20) DEFAULT 'completed',
    location_data JSONB,
    expires_at BIGINT NOT NULL, -- uploadedAt + (48 * 3600 * 1000)
    is_expired BOOLEAN NOT NULL DEFAULT FALSE,
    created_at BIGINT NOT NULL
);

CREATE INDEX attachments_message_idx ON message_attachments (message_id);
CREATE INDEX attachments_expires_idx ON message_attachments (expires_at) WHERE is_expired = FALSE;
```

7. Supporting Tables: Receipts & Outbox Pattern

`message_receipts` Table

```
CREATE TABLE message_receipts (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    message_id UUID NOT NULL REFERENCES messages(id)
ON DELETE CASCADE,
    user_id UUID NOT NULL REFERENCES users(id) ON DE
LETE CASCADE,
    status VARCHAR(20) NOT NULL DEFAULT 'delivered',
    delivered_at BIGINT,
    read_at BIGINT
);

CREATE UNIQUE INDEX receipt_msg_user_idx
    ON message_receipts (message_id, user_id);
```

`outbox_events` Table

```
CREATE TABLE outbox_events (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    event_type VARCHAR(100) NOT NULL,
    payload JSONB NOT NULL,
    status VARCHAR(20) NOT NULL DEFAULT 'pending',
    retry_count INTEGER NOT NULL DEFAULT 0,
    created_at BIGINT NOT NULL,
    published_at BIGINT
);

CREATE INDEX outbox_status_idx
    ON outbox_events (status);
```

8. High-Frequency Production Queries

Query A: Paginated Conversation History with Attached Media

```
-- Retrieves 50 messages strictly ordered with attachments embedded via JSON aggregation
SELECT
    m.id,
    m.conversation_id AS "conversationId",
    m.sender_id AS "senderId",
    u.email AS "senderEmail",
    u.name AS "senderName",
    u.avatar_url AS "senderAvatar",
    m.content,
    m.type,
    m.status,
    m.timestamp_display AS "timestamp",
    m.created_at AS "createdAt",
    m.is_edited AS "isEdited",
    COALESCE(
        jsonb_agg(
            jsonb_build_object(
                'id', a.id,
                'type', a.type,
                'name', a.name,
                'url', a.url,
                'thumbnailUrl', a.thumbnail_url,
                'size', a.size,
                'isExpired', a.is_expired
            )
        ) FILTER (WHERE a.id IS NOT NULL),
        '[]'::jsonb
    ) AS attachments
FROM messages m
INNER JOIN users u ON m.sender_id = u.id
LEFT JOIN message_attachments a ON m.id = a.message_id
WHERE m.conversation_id = $1
  AND m.deleted_for_everyone = FALSE
GROUP BY m.id, u.id
ORDER BY m.created_at ASC
LIMIT 50;
```

Query B: Atomic Read Status Update

```sql
-- Mark messages read and clear unread counter atomically in a transaction
BEGIN;
UPDATE messages
SET status = 'read'
WHERE conversation_id = $1
  AND sender_id != $2
  AND status != 'read';

UPDATE conversations
SET unread_by_client = CASE WHEN $3 = 'client' THEN 0 ELSE unread_by_client END,
    unread_by_business = CASE WHEN $3 = 'business' THEN 0 ELSE unread_by_business END,
    updated_at = clock_timestamp()
WHERE id = $1;
COMMIT;
```

Query C: 48-Hour Media Purge Routine

```sql
-- Select expired media for object store deletion, then flag as expired in DB
UPDATE message_attachments
SET is_expired = TRUE,
    url = '',
    thumbnail_url = NULL
WHERE expires_at <= $1 -- Current Unix epoch in ms
  AND is_expired = FALSE
RETURNING id, object_key;
```